Java, Spring Boot, Microservices & Cloud-Native Backend

The Complete Course: from JVM internals to Kubernetes

7 modules · 24 topics · analogies, diagrams, production code, common mistakes, check-in questions and an answer key

How to Use This Course

Every topic follows the same rhythm, designed so you learn one idea at a time:

1. **Real-world analogy:** build intuition before any code.
2. **Diagram & production-grade code:** see how it actually works.
3. **Common mistake:** what juniors get wrong and how to debug it.
4. **Key takeaways:** the three points to remember.
5. **Check for understanding:** answer it before turning to the Answer Key at the end.

Learning path

Module	Focus	Why it matters
1. Java Core	JVM, GC, collections, threads, virtual threads	The runtime under everything
2. SQL	Indexes, ACID, isolation levels	Correct, fast transactional data
3. MongoDB	Document modelling, embed vs reference	Flexible schemas, read-optimised data
4. Redis	Cache-aside, avalanche/penetration/breakdown	Speed and protecting the database
5. Spring Boot	Auto-config, @Transactional proxies, Resilience4j	The framework you ship with
6. Microservices	Kafka partitions, consumer groups, Saga, outbox	Scaling teams and systems
7. Cloud-Native	Docker multi-stage, Kubernetes, probes, troubleshooting	Running it in production

Versions: examples target **Java 21 (LTS)** and **Spring Boot 3.x**, PostgreSQL 16, MongoDB 7, Redis 7, Kafka 3.x and Kubernetes 1.30+.

Tip: type out the code yourself rather than copy-pasting. Muscle memory matters.

Contents

Module 1: Java Core Internals **4**

1.1 JVM Memory & Garbage Collection . . . 4

1.2 Collections Internals: ArrayList, HashMap, ConcurrentHashMap . . . 5

1.3 Multithreading & Concurrency . . . 7

1.4 Virtual Threads (Java 21+) . . . 8

Module 2: SQL & Relational Databases **10**

2.1 Indexes: B-Trees, Composite Indexes & EXPLAIN . . . 10

2.2 ACID Transactions . . . 11

2.3 Isolation Levels & Concurrency Anomalies . . . 12

Module 3: MongoDB & Document Storage **14**

3.1 The Document Model & When to Use MongoDB . . . 14

3.2 Embedding vs. Referencing . . . 15

3.3 MongoDB Indexes & Aggregation Pipeline . . . 16

Module 4: Redis Caching **18**

4.1 Redis Fundamentals & Data Structures . . . 18

4.2 The Cache-Aside Pattern . . . 19

4.3 Cache Failure Modes: Avalanche, Penetration, Breakdown . . . 20

Module 5: Spring Boot Core & Resiliency **22**

5.1 IoC, Dependency Injection & Auto-Configuration . . . 22

5.2 @Transactional & the Proxy Trap . . . 23

5.3 Resilience4j: Circuit Breaker, Retry, Timeouts & Bulkheads . . . 25

Module 6: Microservices & Event Streaming **27**

6.1 Microservice Architecture Fundamentals . . . 27

6.2 Kafka: Topics, Partitions & Consumer Groups . . . 28

6.3 The Saga Pattern & Transactional Outbox . . . 29

6.4 Delivery Guarantees, Idempotency & Dead-Letter Topics . . . 31

Module 7: Containerization & Kubernetes **32**

7.1 Docker & Multi-Stage Builds . . . 32

7.2 Kubernetes Core Objects: Pod, Deployment, Service, ConfigMap, Secret . . . 33

7.3 Health Probes: Liveness, Readiness & Startup . . . 34

7.4 Troubleshooting Pods . . . 35

Capstone: E-Commerce Order Management System **37**

Answer Key **39**

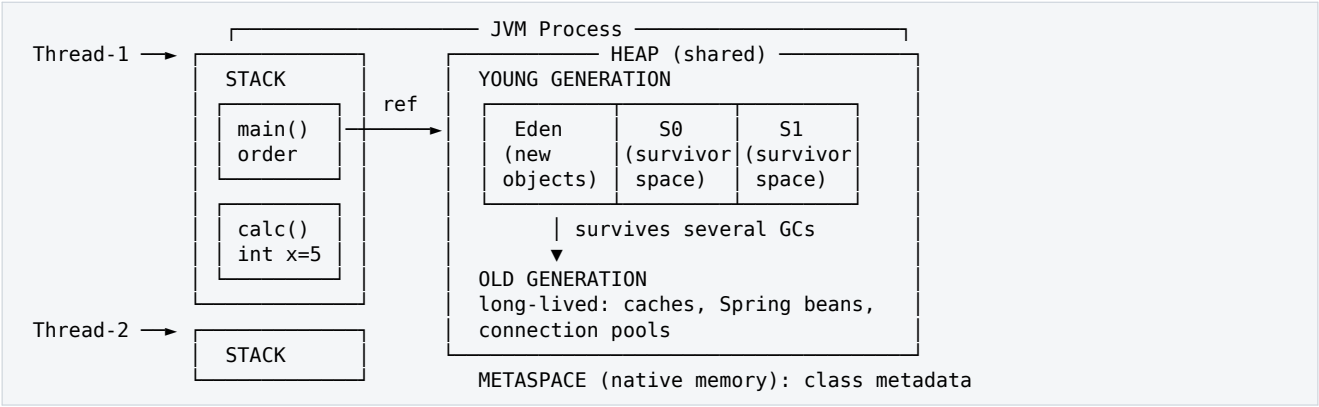
Module 1: Java Core Internals

Everything in the backend stack runs on the JVM. Knowing how memory, collections and threads really work is what separates someone who uses Spring Boot from someone who can debug it at 2 a.m. when production is slow.

1.1 JVM Memory & Garbage Collection

> **Real-world analogy**
>
> The JVM is a **restaurant kitchen**. Each chef has a small private **cutting board** (the Stack) that is wiped clean after each dish. All chefs share one big **pantry** (the Heap). The **recipe book** on the wall is Metaspace. A **cleaning crew** (the Garbage Collector) throws out ingredients nobody can reach anymore. It never touches what a chef is still holding.

Area	What lives there	Lifetime
Stack (one per thread)	Method frames, primitive locals, object references	Freed automatically when the method returns
Heap (shared)	Every object created with new, arrays	Until unreachable, then GC
Metaspace (native)	Class metadata, method bytecode	Until the class loader is unloaded



What goes where?

```
public class OrderService {
    private static final double TAX_RATE = 0.18;   // constant, part of class data

    public Order createOrder(String customerId) {   // new stack frame pushed
        int quantity = 2;                            // STACK: the value itself
        Order order = new Order(customerId);         // reference on STACK, object on HEAP (Eden)
        return order;
    }                                                // frame popped; object survives while referenced
}
```

How the GC decides what to delete

The GC starts from **GC Roots** (local variables on live stacks, static fields, active threads) and follows every reference. Anything it can't reach is garbage.

```
GC Roots ──► Order ──► Customer        ✔ reachable   → kept
             Cart  ──► Item            ✘ unreachable → collected
```

The generational hypothesis

1. New objects are allocated in **Eden**.
2. Eden full → **Minor GC** (fast, only scans young gen). Survivors move to S0/S1.

3. Objects that survive several cycles are **promoted** to **Old Gen**.
4. Old Gen full → **Major / Full GC** (expensive, can pause the application).

Collector	Best for	Flag
G1 (default)	Most Spring Boot services, balanced	`-XX:+UseG1GC`
ZGC	Low-latency APIs, sub-millisecond pauses	`-XX:+UseZGC` (generational by default in Java 23+)
Parallel	Batch jobs, max throughput	`-XX:+UseParallelGC`

```
java -Xms512m -Xmx512m \
     -XX:+UseG1GC -XX:MaxGCPauseMillis=200 \
     -XX:+HeapDumpOnOutOfMemoryError \
     -Xlog:gc*:file=gc.log \
     -jar order-service.jar
```

Common mistake

"Java has GC, so I can't leak memory." Wrong. A `static Map` that you only `put()` into is always reachable from a GC Root, so it grows until `OutOfMemoryError`. Use a bounded cache (Caffeine `maximumSize` / `expireAfterAccess`) or an external store like Redis.

Key takeaways

Objects on the heap, references and primitives on the stack.

GC removes only unreachable objects. Leaks are references you forgot to drop.

Most objects die young: cheap Minor GCs. Frequent Full GCs mean something is wrong.

Check for understanding

Q1. In `for (...) { Order temp = new Order(...); temp.validate(); }`, where does `temp` live and where do the `Order` objects live?

Q2. After each iteration, is the previous `Order` eligible for GC? Why?

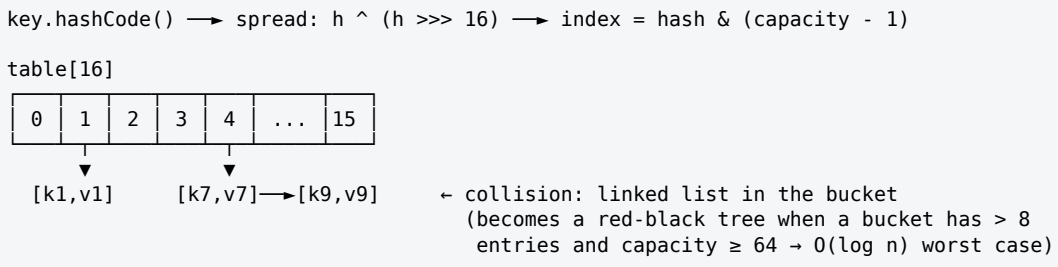
Q3. Will these short-lived objects most likely be cleaned up by a Minor or Major GC?

1.2 Collections Internals: ArrayList, HashMap, ConcurrentHashMap

Real-world analogy

A **HashMap** is a wall of **post-office boxes**. The box number is computed from your name (the hash). To find your mail the clerk computes the number and opens one box instead of searching the whole office. If two people get the same box (a collision), their letters share it and the clerk checks each name (`equals`).

HashMap: how `put(key, value)` works



- **Default capacity 16, load factor 0.75.** When size > capacity × 0.75, the table doubles and every entry is rehashed (expensive). Pre-size big maps: `new HashMap<>(expected * 4 / 3 + 1)`.
- Average `get`/`put` is **O(1)**; it degrades only with bad `hashCode()`s.
- **Contract:** if `a.equals(b)` then `a.hashCode() == b.hashCode()`. Break it and lookups fail silently.

ArrayList vs LinkedList

Operation	ArrayList	LinkedList
`get(i)`	O(1): direct array index	O(n): walk the nodes
Add at end	Amortised O(1), grows ~1.5×	O(1)
Insert in middle	O(n): shift elements	O(n) to find + O(1) to link
Memory / CPU cache	Compact, cache-friendly	Node object per element, scattered

In practice **ArrayList wins almost always**. For queues use `ArrayDeque`.

Concurrent access

`HashMap` is not thread-safe: concurrent writes can lose entries. `Collections.synchronizedMap` locks the whole map. `ConcurrentHashMap` uses CAS and locks only a single bucket, so many threads can write at once.

```
// Thread-safe counter per product, no external locking needed
private final ConcurrentHashMap<String, LongAdder> views = new ConcurrentHashMap<>();

public void recordView(String productId) {
    views.computeIfAbsent(productId, id -> new LongAdder()).increment();
}
```

Common mistake

Using a **mutable object as a HashMap key** and changing it after insertion. Its hashCode changes, so the entry now sits in the "wrong" bucket and `get()` returns `null` even though the entry is still there. Use immutable keys: `String`, `record`, `UUID`.

Key takeaways

HashMap = array of buckets + hashing; good `hashCode`/`equals` are essential.

Default to `ArrayList`, `HashMap`, `ArrayDeque`; `ConcurrentHashMap` for shared state.

Java `record`s give you correct `equals`/`hashCode` for free, so they make ideal keys.

Check for understanding

Q1. You put 1,000,000 entries into a `new HashMap<>()`. Roughly how many resizes happen and how can you avoid them?

Q2. Mini-challenge: write a `record OrderKey(String customerId, LocalDate date)` and explain why it is safe as a map key.

1.3 Multithreading & Concurrency

Real-world analogy

Two cashiers updating **one shared paper ledger**. Both read "balance = 100", both add 10, both write "110". One sale vanished. That is a **race condition**. A `synchronized` block is a **single pen**: only the cashier holding it may write.

The race condition

```
class Counter {
    private int count = 0;
    public void increment() { count++; }   // NOT atomic: read → add → write
}
// 2 threads × 1000 increments → often prints ~1700, not 2000
```

Three fixes, from simplest to most flexible

```
// 1. Atomic variable: lock-free, uses CPU compare-and-swap (CAS)
private final AtomicInteger count = new AtomicInteger();
public void increment() { count.incrementAndGet(); }

// 2. synchronized: mutual exclusion + memory visibility
public synchronized void increment() { count++; }

// 3. Explicit lock: tryLock with timeout, fairness, multiple conditions
private final ReentrantLock lock = new ReentrantLock();
public void increment() {
    lock.lock();
    try { count++; } finally { lock.unlock(); }   // ALWAYS unlock in finally
}
```

`volatile` only guarantees visibility (other threads see the latest value). It does **not** make `count++` atomic. Use it for flags like `volatile boolean running`.

Never create raw threads in a server: use executors

```
ExecutorService pool = Executors.newFixedThreadPool(8);

CompletableFuture<User>    user    = CompletableFuture.supplyAsync(() -> userClient.get(id), pool);
CompletableFuture<List<Order>> orders = CompletableFuture.supplyAsync(() -> orderRepo.find(id), pool);

Dashboard d = user.thenCombine(orders, Dashboard::new)   // run both in parallel, then merge
                  .orTimeout(2, TimeUnit.SECONDS)
                  .join();
```

Deadlock

```
Thread A: holds lock(account1) ──waits for──► lock(account2)
Thread B: holds lock(account2) ──waits for──► lock(account1)      ⇒ both wait forever

Fix: always acquire locks in a global order (e.g. by account id).
```

Common mistake

Sharing a non-thread-safe object across threads, e.g. a `SimpleDateFormat` in a Spring singleton bean. Spring beans are singletons by default, so **every request thread shares them**. Keep beans stateless, or use thread-safe types (`DateTimeFormatter`).

Key takeaways

Shared mutable state + multiple threads = race conditions.

Prefer immutability, then atomics/concurrent collections, then locks.

Use `ExecutorService` / `CompletableFuture`, never `new Thread()` per request.

Check for understanding

Q1. Why doesn't `volatile int count; count++;` fix the race condition?

Q2. Your Spring `@Service` has a field `private List<String> errors = new ArrayList<>();` that each request appends to. What goes wrong under load?

1.4 Virtual Threads (Java 21+)

Real-world analogy

Platform threads are like **restaurant tables**: expensive, so you only have 200. A guest waiting for food still occupies a table. Virtual threads are **restaurant pagers**: cheap and unlimited. A waiting guest just holds a pager while the table (the real OS thread, the carrier) is used by someone who is actually eating.

```
Platform thread model                    Virtual thread model
─────────────────────                    ────────────────────

1 Java thread = 1 OS thread (~1 MB)      millions of virtual threads (~few KB)
                                                 │ mounted only while running
200 threads, 200 blocked on DB                   ▼
 → request #201 waits in queue           ┌──────────────┐
                                         │ carrier threads │  (≈ number of CPU cores)
                                         └──────────────┘
                                         on blocking I/O the virtual thread is
                                         UNMOUNTED, so the carrier serves another one
```

Using them

```
// Plain Java
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<Price>> prices = productIds.stream()
        .map(id -> executor.submit(() -> priceClient.fetch(id)))   // 10,000 tasks? fine
        .toList();
}

# Spring Boot 3.2+  (application.yml): Tomcat & @Async run on virtual threads
spring:
  threads:
    virtual:
      enabled: true
```

Good fit	Poor fit
I/O-bound work: REST calls, DB queries, Kafka, file I/O	CPU-bound work (image processing, crypto): use a bounded platform pool
Thread-per-request servers with high concurrency	Code relying heavily on `ThreadLocal` caches

- **Don't pool virtual threads.** Create one per task; they are cheap.
- **Limit downstream resources, not threads.** 10,000 virtual threads can still exhaust a 20-connection DB pool. Use a `Semaphore` or rely on the pool's timeout.
- **Pinning:** in Java 21–23 a virtual thread blocking inside `synchronized` pins its carrier. Prefer `ReentrantLock` there. Java 24 (JEP 491) removed most pinning.

Common mistake

Turning on virtual threads and expecting CPU-heavy endpoints to get faster. Virtual threads improve **throughput for waiting** (I/O), not raw computation speed.

Key takeaways

Virtual threads = cheap, blocking-style code with async-level scalability.

Keep writing simple synchronous code; the JVM handles the unmounting.

Bottlenecks move to connection pools and downstream services, so watch those.

Check for understanding

Q1. Your service makes 3 slow REST calls (300 ms each) per request and gets 2,000 concurrent requests. Why do virtual threads help here, and what new bottleneck should you watch?

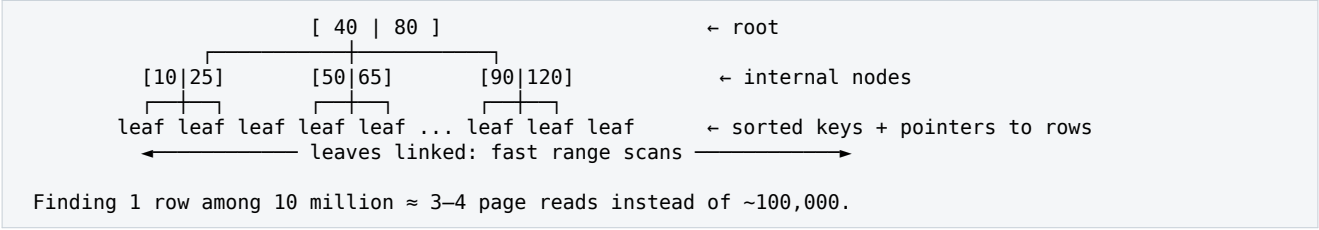
Module 2: SQL & Relational Databases

PostgreSQL is the backbone of most transactional systems: orders, payments, inventory. Three ideas decide whether it is fast and correct under load: **indexes**, **ACID transactions** and **isolation levels**.

2.1 Indexes: B-Trees, Composite Indexes & EXPLAIN

> **Real-world analogy**
>
> An index is the **index at the back of a textbook**. Without it you flip through every page to find "Garbage Collection" (a **sequential scan**). With it you jump to the right page directly. But every time the book is edited, the index must be updated too, so indexes speed up reads and slow down writes.

B-tree: the default index



Creating the right index

```
-- Query the app runs constantly:
SELECT * FROM orders
WHERE customer_id = 42 AND status = 'PAID'
ORDER BY created_at DESC LIMIT 20;

-- Composite index matching filter columns first, then the sort column
CREATE INDEX idx_orders_customer_status_created
    ON orders (customer_id, status, created_at DESC);

-- Verify with the real execution plan
EXPLAIN ANALYZE SELECT ...;
--  Index Scan using idx_orders_customer_status_created on orders
--  (actual time=0.031..0.090 rows=20)        ✔  was: Seq Scan ... (actual time=812 ms)
```

The leftmost-prefix rule

A composite index on `(a, b, c)` is sorted by `a`, then `b`, then `c`, like a phone book sorted by last name, then first name. It can serve queries that use the **leftmost columns**:

Query filters on	Uses index (a, b, c)?
`a`	Yes
`a, b`	Yes
`a, b, c`	Yes
`b` or `c` only	No: like searching a phone book by first name
`a, c`	Partially (only `a` narrows the search)

Things that silently kill an index

- Wrapping the column in a function: `WHERE LOWER(email) = ?`. Fix: an expression index `CREATE INDEX ON users (LOWER(email))`.
- Leading wildcard: `LIKE '%son'` can't use a B-tree (`LIKE 'son%'` can).
- Type mismatch: comparing a `varchar` column to a number forces a cast.

- Low selectivity: an index on `is_active` (two values) is rarely used; the planner prefers a scan.

Common mistake

Missing index on foreign keys. PostgreSQL does not auto-index FK columns. `orders.customer_id` without an index makes every join and every `DELETE` of a customer scan the whole orders table. Rule of thumb: index every FK you join or filter on.

Key takeaways

Index the columns in your `WHERE`, `JOIN` and `ORDER BY`, and order composite columns deliberately.

Always confirm with `EXPLAIN ANALYZE`; never guess.

Each index costs write speed and disk. Drop unused ones (`pg_stat_user_indexes`).

Check for understanding

Q1. You have an index on `(status, customer_id)`. A query filters only by `customer_id`. Will the index be used efficiently? How would you fix it?

2.2 ACID Transactions

Real-world analogy

A **bank transfer** of ₹500 from Asha to Ravi. Taking money from Asha and adding it to Ravi must happen **together or not at all**. If the power fails in between, money must not vanish. That all-or-nothing guarantee is a transaction.

Letter	Guarantee	How Postgres achieves it
Atomicity	All statements succeed or all are rolled back	Undo on `ROLLBACK` or crash
Consistency	Data always satisfies constraints	`PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `NOT NULL`
Isolation	Concurrent transactions don't see each other's half-done work	MVCC snapshots + locks (next topic)
Durability	Once committed, it survives a crash	Write-Ahead Log (WAL) flushed to disk on commit

```
BEGIN;
  UPDATE accounts SET balance = balance - 500 WHERE id = 'asha';
  UPDATE accounts SET balance = balance + 500 WHERE id = 'ravi';
  -- CHECK (balance >= 0) constraint fails?  → whole transaction rolls back
COMMIT;
```

In Spring Boot

```
@Service
@RequiredArgsConstructor
public class TransferService {
    private final AccountRepository accounts;

    @Transactional                                  // BEGIN ... COMMIT / ROLLBACK
    public void transfer(String from, String to, BigDecimal amount) {
        Account source = accounts.findById(from).orElseThrow();
        Account target = accounts.findById(to).orElseThrow();
        source.withdraw(amount);                    // throws InsufficientFundsException
        target.deposit(amount);
    }   // dirty-checking flushes both UPDATEs here; any RuntimeException → rollback
}
```

Keep transactions short

A transaction holds a database connection and row locks until it ends. Long ones block other users and drain the connection pool.

Common mistake

Calling a slow **external API inside** `@Transactional`. The DB connection sits idle for seconds while waiting on HTTP. Under load the Hikari pool (default 10 connections) runs dry: `Connection is not available, request timed out after 30000ms`. Do remote calls before or after the transaction, not inside.

Key takeaways

Atomicity = all-or-nothing; Durability = WAL; Consistency = constraints.

Put business invariants in DB constraints too, not only in Java.

Short transactions; no network calls inside them.

Check for understanding

Q1. Your `placeOrder()` is `@Transactional` and calls the payment gateway (2 s) in the middle. Why is that dangerous, and how would you restructure it?

2.3 Isolation Levels & Concurrency Anomalies

Real-world analogy

Several people editing a **shared Google Sheet**. Do you see others' typing as it happens (dirty), only after they press save (read committed), or a frozen copy from when you opened it (snapshot)? Isolation levels pick which view you get, trading safety against speed.

The anomalies

Anomaly	What happens
Dirty read	You read another transaction's uncommitted change, which is later rolled back
Non-repeatable read	You read a row twice and get different values (someone committed in between)
Phantom read	You run the same `WHERE` query twice and new rows appear
Lost update	Two transactions read-modify-write the same row; one overwrite is lost
Write skew	Two transactions each check a condition, then write different rows, breaking a rule together

The levels (PostgreSQL behaviour)

Level	Dirty	Non-repeatable	Phantom	Notes
Read Uncommitted	No*	Possible	Possible	*Postgres treats it as Read Committed
Read Committed (default)	No	Possible	Possible	Each statement sees a fresh snapshot
Repeatable Read	No	No	No*	One snapshot per transaction; *Postgres prevents phantoms too
Serializable	No	No	No	Also prevents write skew; may abort with a serialization error

Fixing the lost update

```
 T1: SELECT stock → 5            T2: SELECT stock → 5
 T1: UPDATE stock = 5 - 1 = 4    T2: UPDATE stock = 5 - 1 = 4      ⇒ 2 items sold, stock shows 4 ✘
```

```
-- Option A: atomic update, the database does the math
UPDATE products SET stock = stock - 1 WHERE id = :id AND stock > 0;   -- check rows affected

-- Option B: pessimistic lock, others wait for the row
SELECT * FROM products WHERE id = :id FOR UPDATE;
```

```
// Option C: optimistic locking in JPA, best for low-contention updates
@Entity
public class Product {
    @Id private Long id;
    private int stock;
    @Version private long version;    // UPDATE ... SET version = v+1 WHERE id = ? AND version = v
}
// a concurrent change → 0 rows updated → ObjectOptimisticLockingFailureException → retry
```

Common mistake

Raising everything to `SERIALIZABLE` "to be safe" without handling failures. Postgres then aborts conflicting transactions with `could not serialize access`, and your app must **retry** them. Higher isolation = more aborts or waiting, so use it only where the invariant needs it.

Key takeaways

Postgres default is Read Committed: no dirty reads, but lost updates are possible.

Fix lost updates with atomic SQL, `SELECT ... FOR UPDATE`, or `@Version`.

Serializable is the strongest; be ready to retry.

Check for understanding

Q1. Flash sale: 1,000 users buy the last 10 units at the same moment. Which locking strategy would you choose and why: optimistic `@Version` or an atomic `UPDATE ... WHERE stock > 0`?

Module 3: MongoDB & Document Storage

Some data doesn't fit neatly in rows: product catalogues where a phone and a T-shirt have totally different attributes, or content that is always read as one whole. MongoDB stores **documents**, and the most important skill is modelling them around how you read the data.

3.1 The Document Model & When to Use MongoDB

Real-world analogy

SQL is a **filing cabinet with fixed forms**: every customer form has the same boxes, and related info lives in other drawers (tables) linked by ID. MongoDB is a **folder per item**: everything about one product (its specs, images, variants) sits in one folder, and each folder can hold different papers.

```
// One product document (BSON, stored in the "products" collection)
{
  "_id": ObjectId("665f..."),
  "sku": "PHN-IP15-128",
  "name": "iPhone 15",
  "category": "phones",
  "price": { "amount": 79900, "currency": "INR" },
  "attributes": { "storage": "128GB", "color": "Black", "screen": "6.1in" },
  "images": ["https://cdn/.../1.jpg", "https://cdn/.../2.jpg"],
  "variants": [ { "sku": "PHN-IP15-256", "storage": "256GB", "price": 89900 } ]
}
// A T-shirt document in the same collection can have { "size": "M", "fabric": "cotton" } instead
```

Spring Data MongoDB

```
@Document(collection = "products")
public record Product(
        @Id String id,
        @Indexed(unique = true) String sku,
        String name,
        String category,
        Money price,
        Map<String, Object> attributes,          // flexible, per-category attributes
        List<String> images,
        List<Variant> variants) {}

public interface ProductRepository extends MongoRepository<Product, String> {
    List<Product> findByCategoryAndPriceAmountLessThan(String category, long maxPrice);
}
```

Choose PostgreSQL when…	Choose MongoDB when…
Strong relations & joins (orders ↔ payments ↔ customers)	Data is naturally hierarchical and read as a whole (catalogue, CMS, user profile)
Multi-row ACID transactions are central (money, inventory)	Schema varies per record or evolves fast
Complex ad-hoc reporting with SQL	Very high write volume, horizontal sharding needed

MongoDB does support multi-document ACID transactions (since 4.0), but if you need them everywhere, that's a signal the data is relational.

Common mistake

Treating MongoDB like SQL: one collection per "table" and `$lookup` joins everywhere. You get the downsides of both. Design documents around your **query patterns**, not around normalisation.

Key takeaways

Document = self-contained JSON-like record; collections can hold varied shapes.

Model for reads: data accessed together should be stored together.

Polyglot persistence: Postgres for orders, MongoDB for the catalogue is a common pairing.

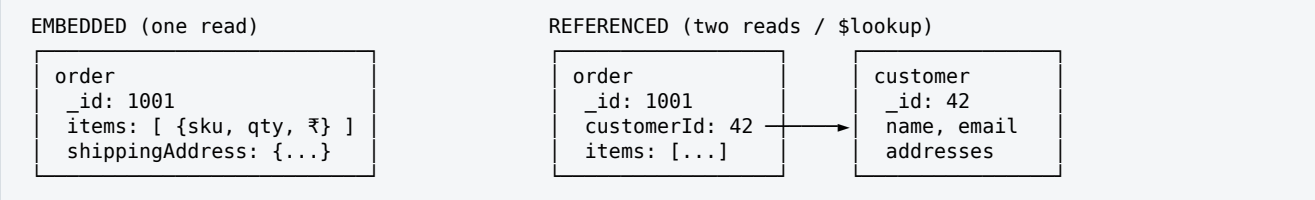
Check for understanding

Q1. For an e-commerce app, which store would you choose for (a) the product catalogue, (b) orders and payments, (c) user shopping-cart sessions? Justify each in one line.

3.2 Embedding vs. Referencing

Real-world analogy

Embedding is stapling the receipt to the order form: one grab gets everything. **Referencing** is writing "see customer file #42" on the form: less duplication, but you need a second trip to the cabinet.



Decision rules

Embed when…	Reference when…
Child belongs to one parent (**contains** relationship)	Child is **shared** by many parents
Data is read together almost always	Child is read/updated independently
Array is **bounded** (an order has ~1–50 items)	Array is **unbounded** (a product's reviews, logs)
A snapshot is wanted (address at time of order)	Data must always be current (live stock)

Useful patterns

- **Extended reference:** store the ID plus the few fields you display, e.g. `customer: { id: 42, name: "Asha" }`. One read for the common screen; full doc when needed.
- **Subset:** embed only the 10 most recent reviews in the product; keep all reviews in a separate `reviews` collection.
- **Bucket:** group time-series events into one document per hour/day instead of one per event.

```
@Document("products")
public record Product(@Id String id, String name,
                      List<Review> recentReviews,      // SUBSET: capped at 10, embedded
                      int reviewCount, double avgRating) {}

@Document("reviews")
public record Review(@Id String id, @Indexed String productId,   // REFERENCE back to product
                     String userId, int rating, String text, Instant createdAt) {}
```

Common mistake

Unbounded arrays. Pushing every review or every event into one document. Documents have a **16 MB hard limit**, and long before that every update rewrites a huge document and memory usage grows. If an array can grow without limit, reference it.

Key takeaways

Embed for contains + read-together + bounded.

Reference for shared, independent or unbounded data.

Duplicating a little data (extended reference) is fine if it saves joins on hot paths.

Check for understanding

Q1. A blog post has comments. Some viral posts get 200,000 comments. The post page shows the latest 20. Design the documents.

3.3 MongoDB Indexes & Aggregation Pipeline

Real-world analogy

The aggregation pipeline is a **factory assembly line**: documents enter, each station (stage) filters, reshapes, groups or sorts them, and a finished report comes out the end.

Compound indexes: the ESR rule

Order fields in a compound index as **E**quality → **S**ort → **R**ange.

```
// Query: category = "phones" AND price BETWEEN 10k AND 50k, sorted by rating desc
db.products.createIndex({ category: 1, rating: -1, "price.amount": 1 })
//                         Equality     Sort         Range

db.products.find({ category: "phones", "price.amount": { $gte: 10000, $lte: 50000 } })
           .sort({ rating: -1 }).explain("executionStats")
// look for: stage IXSCAN (good) vs COLLSCAN (bad); totalDocsExamined ≈ nReturned
```

Aggregation: top categories by revenue

```
db.orders.aggregate([
  { $match:  { status: "DELIVERED", createdAt: { $gte: ISODate("2026-01-01") } } },  // filter early
  { $unwind: "$items" },
  { $group:  { _id: "$items.category",
               revenue: { $sum: { $multiply: ["$items.price", "$items.qty"] } },
               orders:  { $addToSet: "$_id" } } },
  { $project: { revenue: 1, orderCount: { $size: "$orders" } } },
  { $sort:  { revenue: -1 } },
  { $limit: 5 }
])
```

```
// Same thing with Spring Data
Aggregation agg = Aggregation.newAggregation(
    match(Criteria.where("status").is("DELIVERED")),
    unwind("items"),
    group("items.category").sum(ArithmeticOperators.Multiply
            .valueOf("items.price").multiplyBy("items.qty")).as("revenue"),
    sort(Sort.Direction.DESC, "revenue"),
    limit(5));
List<CategoryRevenue> top = mongoTemplate.aggregate(agg, "orders", CategoryRevenue.class)
                                         .getMappedResults();
```

Common mistake

Putting `$match` **after** `$group` or `$unwind`. Indexes can only help stages at the start of the pipeline. Filter first so fewer documents flow down the line.

Key takeaways

Compound index order: Equality, Sort, Range.

Use `explain("executionStats")`: IXSCAN good, COLLSCAN bad.

`$match` and `$project` as early as possible in pipelines.

Check for understanding

Q1. Query: `{ userId: X, status: "ACTIVE" }` sorted by `createdAt` desc, with `createdAt > lastWeek`. Write the ideal compound index.

Module 4: Redis Caching

A database query takes milliseconds; a Redis lookup takes well under one. Caching is the cheapest way to make a system fast, and it is also the easiest way to serve stale data or take down your database when the cache fails. This module covers both sides.

4.1 Redis Fundamentals & Data Structures

Real-world analogy

The database is a **warehouse** across town; Redis is the **shelf behind the counter**. Popular items sit on the shelf for instant pickup. The shelf is small (RAM), so items have expiry labels (TTL) and the shelf may be cleared.

- **In-memory** key-value store: sub-millisecond reads.
- Executes commands on a **single thread**, so each command is atomic (`INCR` never races).
- **Persistence** is optional: RDB snapshots and/or AOF append log.
- **TTL** on any key: `SET session:abc ... EX 1800`.

Type	Example command	Backend use case
String	`SET product:42 '{json}' EX 600`	Cached objects, counters (`INCR`)
Hash	`HSET cart:u7 sku123 2`	Shopping cart, user session fields
List	`LPUSH recent:u7 sku9` + `LTRIM 0 9`	Recently viewed items
Set	`SADD online-users u7`	Unique visitors, tags
Sorted Set	`ZINCRBY bestsellers 1 sku9`	Leaderboards, top-N products
Key + `SET NX`	`SET lock:order:1 me NX EX 10`	Distributed lock (simple)

```
# Rate limiting: max 100 requests per minute per user (fixed window)
INCR   rate:u7:202610211405        → 1, 2, 3 ...
EXPIRE rate:u7:202610211405 60      (set on first hit)
if value > 100 → HTTP 429
```

Spring Boot setup

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

# application.yml
spring:
  data:
    redis:
      host: localhost
      port: 6379
      timeout: 200ms          # fail fast: a slow cache is worse than no cache
```

Common mistake

Running `KEYS *` in production. It scans every key while blocking the single thread, so all clients freeze. Use `SCAN` with a cursor instead. The same goes for huge values ("big keys") and `DEL` on giant collections (use `UNLINK`).

Key takeaways

Redis = fast, in-memory, single-threaded command execution, rich data types.

Always set a TTL on cache keys.

Avoid blocking commands (KEYS, big-key deletes) in production.

Check for understanding

Q1. Which Redis data structure would you use for a "Top 10 best-selling products today" widget, and which commands?

4.2 The Cache-Aside Pattern

Real-world analogy

A **librarian with a desk drawer**. Someone asks for a book: she checks the drawer first (cache). If it's not there, she walks to the stacks (database), hands over the book and drops a copy in the drawer for next time. When a book is revised, she **throws out the drawer copy** rather than trying to edit it.



With Spring Cache annotations

```
@Configuration
@EnableCaching
public class CacheConfig {
    @Bean
    RedisCacheManager cacheManager(RedisConnectionFactory cf) {
        var json = RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer());
        var defaults = RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(10))
                .disableCachingNullValues()
                .serializeValuesWith(json);
        return RedisCacheManager.builder(cf).cacheDefaults(defaults).build();
    }
}

@Service
@RequiredArgsConstructor
public class ProductService {
    private final ProductRepository repo;

    @Cacheable(cacheNames = "products", key = "#id")    // cache-aside read
    public ProductDto get(String id) {
        return repo.findById(id).map(ProductDto::from).orElseThrow();
    }

    @Transactional
    @CacheEvict(cacheNames = "products", key = "#id")   // evict after update
    public ProductDto updatePrice(String id, Money price) { ... }
}
```

Why delete instead of update the cache?

Two concurrent writers can update the DB in order A→B but the cache in order B→A, leaving the cache permanently stale. Deleting is idempotent: the next read loads the truth. The TTL is your safety net for any remaining race.

Common mistake

Caching **JPA entities** directly. Lazy-loaded relations blow up on deserialization (`LazyInitializationException`) or pull half the database into Redis. Cache small, immutable **DTOs/records** instead.

Key takeaways

Read: cache → miss → DB → populate with TTL.

Write: update DB, then **evict** the key.

Cache DTOs, use JSON serialization, always set TTLs.

Check for understanding

Q1. Why should the cache eviction happen after the DB transaction commits, not before? What could go wrong otherwise?

4.3 Cache Failure Modes: Avalanche, Penetration, Breakdown

Real-world analogy

The cache is a **dam** protecting a village (the database). **Avalanche**: the whole dam opens at once. **Penetration**: water seeps through a hole the dam doesn't cover (requests for things that don't exist). **Breakdown**: one critical gate fails and a torrent hits one spot (a single hot key expires).

Failure	Cause	Fixes
Avalanche	Many keys expire at the same moment (all loaded at startup with TTL=10 min), or Redis goes down	Add random **jitter** to TTLs; Redis HA (Sentinel / Cluster); circuit breaker + rate limit on DB fallback
Penetration	Requests for keys that exist in **neither** cache nor DB (`/products/-1`, bots), so every request hits the DB	Validate input; **cache empty results** briefly; **Bloom filter** of valid IDs
Breakdown (stampede)	One **hot key** expires; thousands of concurrent misses all query the DB to rebuild it	**Mutex** so one thread rebuilds; logical expiry / refresh-ahead; never expire hot keys

Fix 1: TTL jitter (avalanche)

```
Duration ttl = Duration.ofMinutes(10)
        .plusSeconds(ThreadLocalRandom.current().nextInt(0, 120));   // spread expiry over 2 min
redis.opsForValue().set(key, value, ttl);
```

Fix 2: cache the "not found" (penetration)

```
private static final String NULL_MARKER = "__NULL__";

public Optional<ProductDto> find(String id) {
    String cached = redis.opsForValue().get("product:" + id);
    if (NULL_MARKER.equals(cached)) return Optional.empty();          // known miss, skip DB
    if (cached != null) return Optional.of(read(cached));

    Optional<ProductDto> fromDb = repo.findById(id).map(ProductDto::from);
    redis.opsForValue().set("product:" + id,
            fromDb.map(this::write).orElse(NULL_MARKER),
            fromDb.isPresent() ? Duration.ofMinutes(10) : Duration.ofMinutes(1));
    return fromDb;
}
```

Fix 3: mutex rebuild (breakdown)

```
public ProductDto getHot(String id) {
    String key = "product:" + id;
    for (int attempt = 0; attempt < 50; attempt++) {
        String cached = redis.opsForValue().get(key);
        if (cached != null) return read(cached);

        String lockKey = "lock:" + key, token = UUID.randomUUID().toString();
        Boolean locked = redis.opsForValue().setIfAbsent(lockKey, token, Duration.ofSeconds(5));
        if (Boolean.TRUE.equals(locked)) {
            try {
                ProductDto dto = ProductDto.from(repo.findById(id).orElseThrow());
                redis.opsForValue().set(key, write(dto), Duration.ofMinutes(10));
                return dto;
            } finally {
                // release only if we still own it (use a Lua script for atomic check-and-delete)
                if (token.equals(redis.opsForValue().get(lockKey))) redis.delete(lockKey);
            }
        }
        sleepQuietly(50);   // someone else is rebuilding; wait briefly and re-check the cache
    }
    throw new ServiceUnavailableException("cache rebuild timed out");
}
```

Common mistake

Assuming the cache is always up. If Redis is down and every request falls through to Postgres, the DB collapses too. Set a short Redis **timeout**, wrap calls in a **circuit breaker** (Module 5) and decide deliberately: degrade, serve stale, or rate-limit.

Key takeaways

Avalanche → jitter TTLs + HA. Penetration → cache nulls / Bloom filter.

Breakdown → one rebuilder (mutex) or refresh-ahead for hot keys.

Design for Redis being down: timeouts and circuit breakers.

Check for understanding

Q1. At midnight, a script warms the cache with 1M product keys, all with TTL = 24h. What happens the next midnight, which failure mode is it, and how do you fix it in one line of code?

Module 5: Spring Boot Core & Resiliency

Spring Boot feels like magic until it misbehaves. This module removes the magic: how beans are wired, how auto-configuration decides what to create, why `@Transactional` sometimes silently does nothing, and how Resilience4j keeps one slow service from taking down the rest.

5.1 IoC, Dependency Injection & Auto-Configuration

> **Real-world analogy**
>
> **Inversion of Control** is a **hotel**: you don't build your own bed or hire your own cleaner; you declare what you need and the hotel provides it. **Auto-configuration** is a **smart hotel** that sees you brought a surfboard and automatically books you a beach locker, unless you've already booked one yourself.

Constructor injection (the right way)

```
@Service
public class OrderService {
    private final OrderRepository orders;          // final → immutable, never null
    private final PaymentClient payments;

    public OrderService(OrderRepository orders, PaymentClient payments) {   // single ctor: no @Autowired needed
        this.orders = orders;
        this.payments = payments;
    }
}
// Testable without Spring: new OrderService(mockRepo, mockClient)
```

What @SpringBootApplication really is

```
@SpringBootApplication  =  @SpringBootConfiguration   (this class can define @Beans)
                         + @ComponentScan             (find @Component/@Service/... in this package & below)
                         + @EnableAutoConfiguration   (load auto-config classes, below)

 spring-boot-autoconfigure.jar
   META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
     ├─ DataSourceAutoConfiguration
     ├─ RedisAutoConfiguration
     ├─ KafkaAutoConfiguration   ...  (~150 candidates, each guarded by conditions)
```

```
// Simplified version of what Boot ships:
@AutoConfiguration
@ConditionalOnClass(DataSource.class)                // only if JDBC is on the classpath
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean                        // back off if YOU defined a DataSource
    DataSource dataSource(DataSourceProperties props) {
        return props.initializeDataSourceBuilder().type(HikariDataSource.class).build();
    }
}
```

Debug it: run with `--debug` (or `debug=true`) to print the **Conditions Evaluation Report**: which auto-configs matched, which didn't, and why. The `/actuator/conditions` endpoint shows the same at runtime.

Type-safe configuration

```
@ConfigurationProperties(prefix = "orders")
@Validated
public record OrderProperties(@NotNull Duration paymentTimeout, @Min(1) int maxItems) {}

# application.yml
orders:
  payment-timeout: 3s
  max-items: 50
```

Common mistake

Field injection (`@Autowired private Repo repo;`). It hides dependencies, allows `null` in unit tests, and makes circular dependencies easy to create. Spring Boot 2.6+ fails on circular references by default, and constructor injection exposes the cycle so you redesign instead of hiding it.

Key takeaways

Constructor injection with `final` fields, always.

Auto-config = classpath + conditions + "back off if user defined it".

`--debug` shows why a bean was or wasn't created.

Check for understanding

Q1. You define your own `@Bean ObjectMapper` with custom settings. Why does Spring Boot's default `ObjectMapper` not get created as well?

5.2 @Transactional & the Proxy Trap

Real-world analogy

`@Transactional` works through a **receptionist** (a proxy). Visitors who come through the front desk get signed in and out (BEGIN/COMMIT). But if someone already inside the building walks to another office down the hall, **the receptionist never sees them**, so nothing gets logged.

```
  Controller ──calls──► [ OrderService$$SpringCGLIB PROXY ] ──► real OrderService
                          │ 1. get connection, BEGIN
                          │ 2. invoke real method
                          │ 3. COMMIT, or ROLLBACK on RuntimeException
                          ▼
  Inside the real object, `this.otherMethod()` bypasses the proxy entirely.
```

Pitfall 1: self-invocation

```
@Service
public class OrderService {
    public void placeOrders(List<OrderRequest> reqs) {
        reqs.forEach(this::placeOne);        // ✘ `this` = real object → @Transactional IGNORED
    }

    @Transactional
    public void placeOne(OrderRequest r) { ... }
}

// ✔ Fix: move placeOne() to another bean (cleanest), or use TransactionTemplate:
private final TransactionTemplate tx;
public void placeOrders(List<OrderRequest> reqs) {
    reqs.forEach(r -> tx.executeWithoutResult(status -> placeOne(r)));
}
```

Pitfall 2: checked exceptions don't roll back

```
@Transactional                                    // rolls back on RuntimeException and Error only
public void importFile(...) throws IOException {
    repo.save(a);
    parser.parse(file);                          // throws IOException → transaction COMMITS a! ✘
}

@Transactional(rollbackFor = Exception.class)    // ✔ explicit
```

Pitfall 3: swallowing the exception

```
@Transactional
public void pay(...) {
    try { ledger.debit(...); }
    catch (Exception e) { log.error("failed", e); }   // ✘ proxy never sees the exception → COMMIT
}
```

Propagation in one table

Propagation	Behaviour	Typical use
`REQUIRED` (default)	Join existing transaction or start one	Almost everything
`REQUIRES_NEW`	Suspend outer, run in a brand-new transaction	Audit log that must persist even if outer rolls back
`MANDATORY`	Fail if no transaction exists	Repository-style methods that must be called inside one
`NOT_SUPPORTED`	Suspend transaction, run without	Long read-only reports / external calls

Use `@Transactional(readOnly = true)` on query methods: Hibernate skips dirty checking and the driver can route to replicas.

Common mistake

Putting `@Transactional` on a **private** method or calling it from the same class and "testing" it by eye. Verify by enabling `logging.level.org.springframework.transaction.interceptor=TRACE`: you will see "Getting transaction for [...]" only when the proxy is actually involved.

Key takeaways

Transactions are applied by a proxy, so only calls through the proxy count.

Default rollback: unchecked exceptions only. Use `rollbackFor` for checked ones.

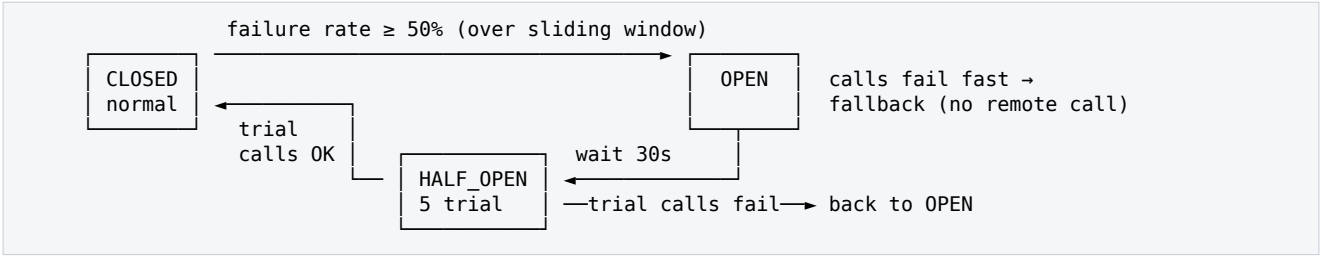
Don't swallow exceptions inside a transactional method.

Check for understanding

Q1. `AuditService.log()` is annotated `@Transactional(propagation = REQUIRES_NEW)` and is called from `OrderService.place()` (a separate bean). `place()` then throws. Is the audit row saved? What if `log()` were a method inside `OrderService`?

5.3 Resilience4j: Circuit Breaker, Retry, Timeouts & Bulkheads

> **Real-world analogy**
>
> A **circuit breaker** is the **electrical fuse** in your house. When a faulty appliance draws too much current, the fuse trips and cuts power to that circuit, protecting the whole house from fire. After a while you try switching it back on; if the fault persists, it trips again.



Setup

```
<!-- pom.xml -->
<dependency><groupId>io.github.resilience4j</groupId>
            <artifactId>resilience4j-spring-boot3</artifactId></dependency>
<dependency><groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-aop</artifactId></dependency>

# application.yml
resilience4j:
  circuitbreaker:
    instances:
      payment:
        sliding-window-size: 20
        minimum-number-of-calls: 10
        failure-rate-threshold: 50
        slow-call-duration-threshold: 2s
        slow-call-rate-threshold: 50
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 5
  retry:
    instances:
      payment:
        max-attempts: 3
        wait-duration: 200ms
        enable-exponential-backoff: true
        retry-exceptions: [java.io.IOException, java.util.concurrent.TimeoutException]
```

```
@Service
@RequiredArgsConstructor
public class PaymentGateway {
    private final PaymentClient client;

    @Retry(name = "payment")
    @CircuitBreaker(name = "payment", fallbackMethod = "paymentUnavailable")
    public PaymentResult charge(ChargeRequest req) {
        return client.charge(req);        // HTTP client MUST have its own connect/read timeouts
    }

    // same signature + the exception as last parameter
    private PaymentResult paymentUnavailable(ChargeRequest req, Throwable t) {
        return PaymentResult.pending(req.orderId(), "Payment service busy, will retry");
    }
}
```

Pattern	Protects against	Analogy
Timeout	Hanging forever on a slow dependency	Hanging up after 10 rings
Retry (with backoff)	Brief, transient glitches	Redialling a dropped call

Pattern	Protects against	Analogy
Circuit breaker	Hammering a service that is down	Electrical fuse
Bulkhead	One slow dependency eating all threads	Ship's watertight compartments
Rate limiter	Overwhelming a dependency or yourself	Club bouncer

Default aspect order: `Retry ( CircuitBreaker ( RateLimiter ( TimeLimiter ( Bulkhead ( call ) ) ) ) )`. Each retry attempt is counted by the circuit breaker, and once it opens retries fail fast.

Common mistake

Retrying non-idempotent calls (e.g. "charge card") without an idempotency key, so the customer gets charged twice when the first call actually succeeded but timed out. Always send an `Idempotency-Key` header, or retry only safe operations.

Key takeaways

Every remote call needs a timeout. That comes before any other pattern.

Circuit breaker: CLOSED → OPEN (fail fast) → HALF_OPEN (probe).

Retries must use backoff and be idempotent; fallbacks should degrade gracefully.

Check for understanding

Q1. The inventory service is down. Without a circuit breaker, your order service has 200 Tomcat threads and each inventory call waits 30 s to time out. Describe what happens to your service, and how the circuit breaker changes it.

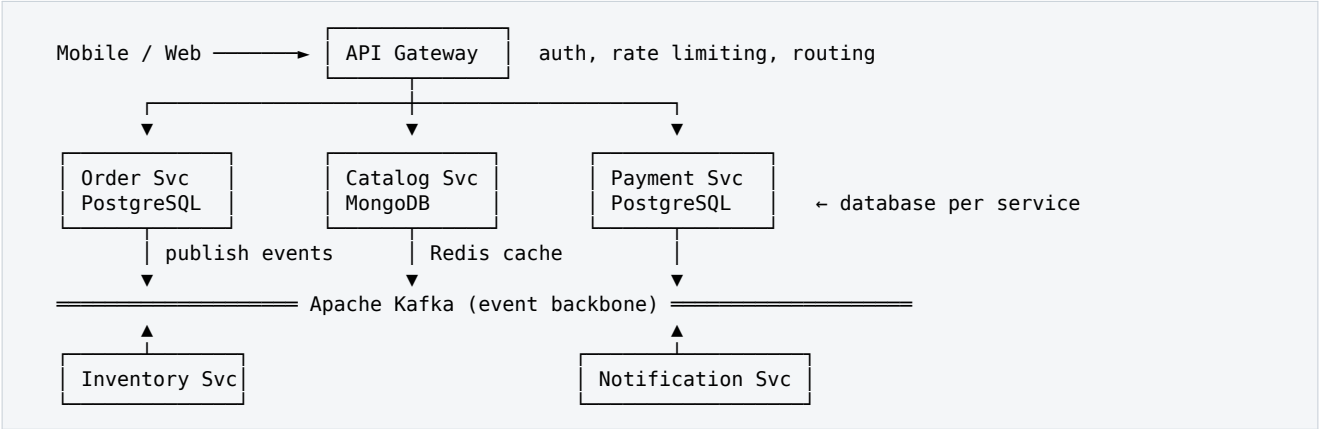
Module 6: Microservices & Event Streaming

Microservices let teams ship independently, but they turn method calls into network calls and one database transaction into many. Kafka and the Saga pattern are the standard tools for keeping a distributed system consistent and decoupled.

6.1 Microservice Architecture Fundamentals

Real-world analogy

A monolith is **one big restaurant kitchen**: easy to coordinate, but one grease fire closes everything. Microservices are a **food court**: each stall (service) has its own kitchen and pantry (database), stalls can open, close or renovate independently, but customers now walk between stalls and stalls must talk.



Core principles

- **Database per service.** No other service reads your tables; they call your API or consume your events.
- **Bounded contexts** (from DDD): split by business capability (Orders, Payments), not by technical layer ("DB service").
- **Sync vs async:** use REST/gRPC when you need an answer now (price lookup); use events when others just need to know something happened (OrderPlaced).
- **Observability is mandatory:** correlation IDs, distributed tracing (OpenTelemetry), centralised logs and metrics.

	Synchronous (REST)	Asynchronous (Kafka events)
Coupling	Caller depends on callee being up	Producer doesn't know consumers
Latency	Adds up along the call chain	Caller returns immediately
Consistency	Immediate	Eventual
Failure	Cascades without circuit breakers	Events wait in Kafka until consumers recover

Common mistake

The **distributed monolith**: services that share one database, or must all be deployed together, or form long synchronous chains (A→B→C→D). You pay the cost of microservices without the benefits. Start with a well-modularised monolith and split when a boundary is clear.

Key takeaways

Each service owns its data and can be deployed independently.

Prefer events for propagating state changes; sync calls for queries needing an answer.

Microservices are an organisational scaling tool. Don't start with 20.

Check for understanding

Q1. After an order is placed you must: reserve stock, send a confirmation email, and update a sales dashboard. Which should be synchronous calls and which async events? Why?

6.2 Kafka: Topics, Partitions & Consumer Groups

Real-world analogy

A Kafka **topic** is a **bank** with several **teller queues** (partitions). Each customer (message) with the same account number (key) always joins the **same queue**, so their transactions are processed in order. A **consumer group** is the team of tellers: each queue is served by exactly one teller, but one teller can handle several queues. Hire more tellers than queues and the extras stand idle.

```
Topic "orders" (3 partitions)                  Consumer group "inventory-svc"
──────────────────────────────                 ──────────────────────────────

P0: [o1][o4][o7][o9] ──────────────────────►  Consumer A  (P0)
P1: [o2][o5][o8]     ──────────────────────►  Consumer B  (P1, P2)
P2: [o3][o6]         ──────────────────────┘

partition = hash(key) % 3     → same orderId ⇒ same partition ⇒ ordered
offsets: each group tracks its own position per partition (P0 @ 4, P1 @ 2 ...)

Another group "notification-svc" reads the SAME messages independently (pub/sub).
```

Concept	Rule to remember
Ordering	Guaranteed **only within a partition**, so pick the key that needs ordering (orderId, customerId)
Parallelism	Max active consumers in a group = **number of partitions**
Retention	Messages stay for the retention period even after being read; consumers can replay
Rebalance	When consumers join/leave/crash, partitions are reassigned; consumption pauses briefly

Producer (Spring Kafka)

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      acks: all                         # wait for all in-sync replicas
      properties:
        enable.idempotence: true        # no duplicates on producer retry
    consumer:
      group-id: inventory-svc
      auto-offset-reset: earliest
      enable-auto-commit: false         # Spring commits after the listener succeeds

@Service
@RequiredArgsConstructor
public class OrderEventPublisher {
    private final KafkaTemplate<String, OrderPlaced> kafka;

    public void publish(OrderPlaced event) {
        kafka.send("orders", event.orderId(), event);      // key = orderId → ordering per order
    }
}
```

Consumer

```
@Component
@RequiredArgsConstructor
public class InventoryListener {
    private final InventoryService inventory;

    @KafkaListener(topics = "orders", concurrency = "3")    // 3 consumer threads ≤ 3 partitions
    public void onOrderPlaced(OrderPlaced event) {
        inventory.reserve(event.orderId(), event.items());  // must be idempotent (see 6.4)
    }
}
```

Common mistake

Endless rebalancing. A listener does slow work (e.g. 10 min batch) and exceeds `max.poll.interval.ms` (default 5 min). Kafka assumes the consumer is dead, rebalances, another consumer re-reads the same messages, also times out... Fix: keep processing fast, reduce `max.poll.records`, or increase the interval. Watch logs for "Revoking partitions" / "rebalance".

Key takeaways

Key → partition → ordering. No key = round-robin, no ordering.

Partitions cap parallelism; plan partition count for future scale.

Each consumer group gets every message; consumers within a group share partitions.

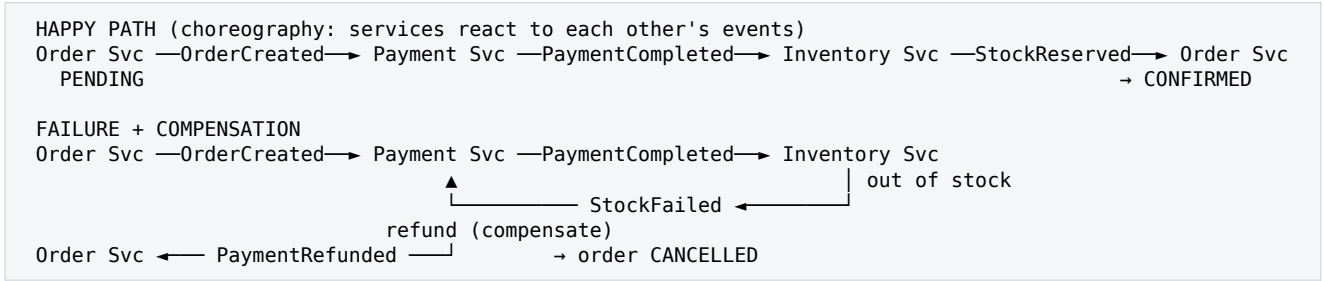
Check for understanding

Q1. A topic has 4 partitions. You scale your consumer service to 6 pods (same group). How many pods actually consume? What would you change to use all 6?

6.3 The Saga Pattern & Transactional Outbox

Real-world analogy

Booking a **holiday**: flight, hotel and car from three different companies. There's no single "undo" button across companies. If the car rental fails, you **cancel** the hotel and the flight yourself. A Saga is a sequence of local transactions where each step has a **compensating action** that undoes it.



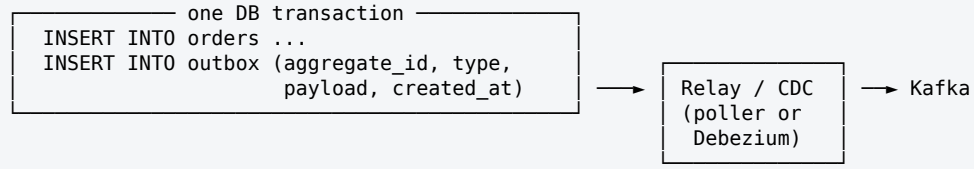
	Choreography	Orchestration
How	Each service listens & reacts to events	A central orchestrator sends commands step by step
Pros	Loose coupling, simple for 2–4 steps	Flow is explicit in one place; easier to monitor/change
Cons	Flow is implicit, hard to trace as it grows	Orchestrator is extra component; risk of "god service"

	Choreography	Orchestration
Tools	Kafka + listeners	Temporal, Camunda, or a custom state machine

The dual-write problem

```
@Transactional
public void createOrder(...) {
    orderRepo.save(order);                       // 1. DB write
    kafka.send("orders", new OrderCreated(...)); // 2. Kafka write ✘
}
// Kafka succeeds but DB rolls back → phantom event.  DB commits but app crashes → lost event.
```

Fix: Transactional Outbox




```
@Transactional
public Order createOrder(CreateOrder cmd) {
    Order order = orderRepo.save(Order.pending(cmd));
    outboxRepo.save(OutboxEvent.of("Order", order.getId(), "OrderCreated", toJson(order)));
    return order;                      // both rows commit or neither does
}

@Scheduled(fixedDelay = 1000)        // simple poller; Debezium CDC is the production-grade option
@Transactional
public void relay() {
    for (OutboxEvent e : outboxRepo.findTop100ByPublishedFalseOrderByIdAsc()) {
        kafka.send("orders", e.aggregateId(), e.payload()).join();
        e.markPublished();
    }
}
```

Common mistake

Forgetting that compensations are **business actions, not rollbacks**. You can't "un-send" an email; you send a cancellation. Design each step's compensation up front, and make it idempotent, because it may run more than once.

Key takeaways

Saga = local transactions + compensating actions; eventual consistency.

Choreography for small flows, orchestration for complex ones.

Use the Outbox pattern to atomically save state and publish events.

Check for understanding

Q1. Mini-challenge: list the Saga steps and compensations for "Place Order" involving Order, Payment and Inventory services. Which step should be last, and why?

6.4 Delivery Guarantees, Idempotency & Dead-Letter Topics

Real-world analogy

A **courier who might ring twice**. If they're unsure you received the parcel, they deliver again. Your job is to check the tracking number and say "already got it, thanks" instead of paying twice. That's an **idempotent consumer**.

Guarantee	Meaning	Reality
At-most-once	Commit offset before processing; may lose messages	Rarely acceptable
At-least-once	Commit after processing; may get duplicates	**The practical default**
Exactly-once	Kafka transactions (read-process-write within Kafka)	Doesn't cover side effects outside Kafka (DB, email, HTTP)

Idempotent consumer

```
CREATE TABLE processed_events (event_id UUID PRIMARY KEY, processed_at TIMESTAMPTZ NOT NULL);

@KafkaListener(topics = "payments")
@Transactional
public void onPaymentCompleted(PaymentCompleted e) {
    if (processedRepo.existsById(e.eventId())) return;          // duplicate → ignore
    orderService.markPaid(e.orderId());
    processedRepo.save(new ProcessedEvent(e.eventId(), Instant.now()));
    // same DB transaction: both happen or neither. The PK also blocks concurrent duplicates.
}
```

Retries and Dead-Letter Topics

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template);         // → "<topic>-dlt"
    var backoff = new ExponentialBackOffWithMaxRetries(3);
    backoff.setInitialInterval(500); backoff.setMultiplier(2.0);
    var handler = new DefaultErrorHandler(recoverer, backoff);
    handler.addNotRetryableExceptions(ValidationException.class);       // poison: straight to DLT
    return handler;
}
```

Common mistake

A **poison message** (malformed JSON, invalid data) with infinite retries blocks the whole partition: every message behind it waits forever. Always configure bounded retries + a DLT, alert on DLT growth, and build a way to inspect and replay it.

Key takeaways

Assume at-least-once: every consumer must handle duplicates.

Idempotency via unique event IDs, upserts or natural keys.

Bounded retries with backoff, then a dead-letter topic.

Check for understanding

Q1. Your consumer charges a loyalty-points bonus on `OrderDelivered`. The pod crashes after adding points but before the offset commit. What happens on restart, and how do you prevent double points?

Module 7: Containerization & Kubernetes

"Works on my machine" ends here. Docker packages your service with everything it needs; Kubernetes runs many copies, restarts them when they fail, and routes traffic only to healthy ones, if you configure it correctly.

7.1 Docker & Multi-Stage Builds

> **Real-world analogy**
> A Docker image is a **shipping container**: whatever is inside (your JAR, the JRE, config), every ship and port handles it the same way. A **multi-stage build** is like building furniture in a **workshop**, then shipping only the finished table, not the saws and sawdust.

```
# syntax=docker/dockerfile:1
# ---------- Stage 1: build (JDK + Maven, ~500 MB, thrown away) ----------
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN ./mvnw dependency:go-offline -B           # cached layer: only reruns when pom.xml changes
COPY src/ src/
RUN ./mvnw package -DskipTests -B \
 && java -Djarmode=tools -jar target/*.jar extract --layers --launcher --destination extracted

# ---------- Stage 2: runtime (JRE only, ~200 MB) ----------
FROM eclipse-temurin:21-jre
WORKDIR /app
RUN groupadd --system spring && useradd --system --gid spring spring
USER spring                                    # never run as root
COPY --from=build /app/extracted/dependencies/ ./
COPY --from=build /app/extracted/spring-boot-loader/ ./
COPY --from=build /app/extracted/snapshot-dependencies/ ./
COPY --from=build /app/extracted/application/ ./   # changes most often → last layer
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "org.springframework.boot.loader.launch.JarLauncher"]
```

Why the layers matter

```
 Layer                    Changes when…          Size
 dependencies             you change pom.xml     ~80 MB   ← cached across most builds
 spring-boot-loader       Boot version upgrade   <1 MB
 snapshot-dependencies    SNAPSHOT deps change   small
 application              every code change      ~200 KB  ← only this is rebuilt & pushed
```

```
# .dockerignore: keep the build context small and secrets out
target/
.git/
*.log
.env
```

`-XX:MaxRAMPercentage=75.0` makes the JVM size its heap from the **container's** memory limit, leaving room for metaspace, threads and native memory. Alternative: Spring Boot's `./mvnw spring-boot:build-image` builds an optimised image with Buildpacks, no Dockerfile needed.

> **Common mistake**
> Setting `-Xmx` equal to the container memory limit (e.g. both 1 GB). The JVM uses memory beyond the heap (metaspace, thread stacks, direct buffers), so the container exceeds its limit and the kernel kills it: **OOMKilled, exit code 137**, with no Java stack trace.

Key takeaways

Multi-stage: build with JDK, run with JRE, so the image is small and safer.

Order layers from least to most frequently changing.

Non-root user; size the heap with `MaxRAMPercentage`, not a hard-coded `-Xmx`.

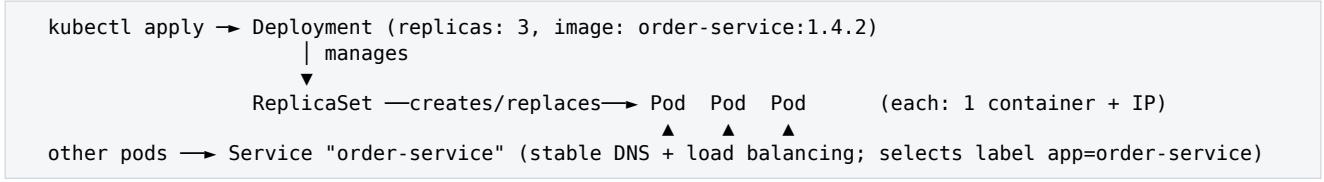
Check for understanding

Q1. Why do we `COPY pom.xml` and download dependencies before `COPY src/`? What happens to build time if you reverse the order?

7.2 Kubernetes Core Objects: Pod, Deployment, Service, ConfigMap, Secret

Real-world analogy

Kubernetes is a **restaurant manager**. You don't tell staff "go to table 4"; you declare "I want 3 waiters on shift at all times" (desired state). If one goes home sick, the manager calls in a replacement. The **Service** is the restaurant's single phone number that rings whichever waiter is free.




```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3
  selector:
    matchLabels: { app: order-service }
  strategy:
    rollingUpdate: { maxUnavailable: 0, maxSurge: 1 }      # zero-downtime deploys
  template:
    metadata:
      labels: { app: order-service }
    spec:
      containers:
        - name: app
          image: registry.example.com/order-service:1.4.2   # pin versions, never :latest
          ports: [{ containerPort: 8080 }]
          envFrom:
            - configMapRef: { name: order-service-config }
            - secretRef:    { name: order-service-secrets }
          resources:
            requests: { cpu: "500m", memory: "768Mi" }      # used for scheduling
            limits:   { memory: "768Mi" }                   # exceed → OOMKilled
---
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector: { app: order-service }
  ports: [{ port: 80, targetPort: 8080 }]
# other pods call http://order-service/api/orders
```

```
apiVersion: v1
kind: ConfigMap
metadata: { name: order-service-config }
data:
  SPRING_PROFILES_ACTIVE: "prod"
  SPRING_DATASOURCE_URL: "jdbc:postgresql://orders-db:5432/orders"
---
# Secret values are only base64-encoded, not encrypted. Restrict access with RBAC and
# enable encryption at rest, or use an external secrets manager (Vault, cloud KMS).
kubectl create secret generic order-service-secrets --from-literal=SPRING_DATASOURCE_PASSWORD=...
```

Common mistake

No resource requests/limits. The scheduler packs pods blindly, one memory-hungry pod starves its neighbours, and the node evicts random pods. Always set requests; set a memory limit; be careful with CPU limits (they cause throttling and slow JVM start-up).

Key takeaways

Deployment = desired state; ReplicaSet/Pods are managed for you.

Service = stable name + load balancing over pods selected by labels.

Config in ConfigMaps, credentials in Secrets, resources always declared.

Check for understanding

Q1. You delete one of the 3 pods with `kubectl delete pod order-service-abc`. What happens next, and which object makes it happen?

7.3 Health Probes: Liveness, Readiness & Startup

Real-world analogy

A **hospital check-up**. Startup probe: "has the patient woken up from surgery yet?" (be patient). Readiness: "can they receive visitors right now?" (if not, send visitors elsewhere, but don't panic). Liveness: "is the heart beating?" (if not, resuscitate, i.e. restart).

Probe	Question	On failure
startupProbe	Has the app finished starting?	Keep waiting until threshold, then restart; other probes are paused
readinessProbe	Can it serve traffic now?	Removed from Service endpoints. **No restart**
livenessProbe	Is it stuck/deadlocked beyond repair?	Container is **restarted**

Spring Boot Actuator support

```
# application.yml (auto-enabled when running on Kubernetes; explicit here for clarity)
management:
  endpoint:
    health:
      probes:
        enabled: true
      group:
        readiness:
          include: readinessState, db, redis     # dependencies affect READINESS only
  endpoints:
    web:
      exposure:
        include: health, info, prometheus
```

```
          startupProbe:
            httpGet: { path: /actuator/health/liveness, port: 8080 }
            periodSeconds: 5
            failureThreshold: 30                  # up to 150 s to start
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8080 }
            periodSeconds: 10
            failureThreshold: 3
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
            periodSeconds: 5
            failureThreshold: 3
```

Also add graceful shutdown so in-flight requests finish during rolling updates: `server.shutdown: graceful` and `spring.lifecycle.timeout-per-shutdown-phase: 20s`.

Common mistake

Checking the database in the liveness probe. The DB has a 30-second blip → liveness fails on every pod → Kubernetes restarts them all at once → all pods are starting simultaneously and hammer the recovering DB. A self-inflicted outage. External dependencies belong in **readiness**, never liveness.

Key takeaways

Liveness = restart me; readiness = don't send me traffic; startup = give me time.

Liveness checks only the app itself; dependencies go in readiness.

Use a startup probe for slow JVM boots instead of a huge liveness delay.

Check for understanding

Q1. A pod takes 90 s to start. Its liveness probe has `initialDelaySeconds: 30`, `periodSeconds: 10`, `failureThreshold: 3`, and no startup probe. What happens, and how do you fix it?

7.4 Troubleshooting Pods

Real-world analogy

Debugging a pod is like a **doctor's diagnosis**: look at the symptoms (status), read the chart (`describe` events), listen to the patient (`logs`), then examine directly (`exec`). Don't prescribe before you diagnose.

```
kubectl get pods  →  what is the STATUS?
│
├─ Pending ──────────────────► kubectl describe pod → Events:
│                               "Insufficient memory/cpu"  → lower requests or add nodes
│                               "unbound PersistentVolumeClaim" → fix storage
│
├─ ImagePullBackOff ─────────► wrong image name/tag, or missing imagePullSecrets for private registry
│
├─ CrashLoopBackOff ─────────► kubectl logs <pod> --previous   (logs of the crashed attempt!)
│                               app exception on startup? bad config/env? liveness killing it?
│
├─ OOMKilled (exit 137) ─────► describe → "Last State: Terminated, Reason: OOMKilled"
│                               raise memory limit or lower MaxRAMPercentage; check for leaks
│
└─ Running but 0/1 READY ────► readiness failing → curl /actuator/health/readiness from inside;
                                Service has no endpoints: kubectl get endpoints <svc>
```

Your toolkit

```
kubectl get pods -o wide                          # status, restarts, node, IP
kubectl describe pod order-service-7d9f-abcde     # events: scheduling, pulls, probe failures, OOM
kubectl logs order-service-7d9f-abcde -f          # live logs
kubectl logs order-service-7d9f-abcde --previous # logs from the container that crashed
kubectl get events --sort-by=.lastTimestamp       # cluster-wide recent events
kubectl exec -it order-service-7d9f-abcde -- sh   # shell inside (if image has one)
kubectl top pod                                   # live CPU/memory (needs metrics-server)
kubectl port-forward svc/order-service 8080:80    # hit the service from your laptop
kubectl rollout undo deployment/order-service     # fastest fix for a bad release
```

Exit code	Meaning
0	Process exited normally (did your main thread finish? a CommandLineRunner app?)
1	Application error: read the logs
137	SIGKILL: usually **OOMKilled**, or failed liveness probe
143	SIGTERM: normal shutdown requested (rolling update, scale-down)

Common mistake

Running `kubectl logs` on a CrashLoopBackOff pod and seeing only a few startup lines, because you're looking at the new attempt. The real error is in `kubectl logs --previous`. And always read the **Events** section of `describe` first: it explains most Pending/ImagePull/OOM problems in one line.

Key takeaways

Status tells you where to look: Pending → scheduling; BackOff → image or app.

`describe` (events) → `logs --previous` → `exec`, in that order.

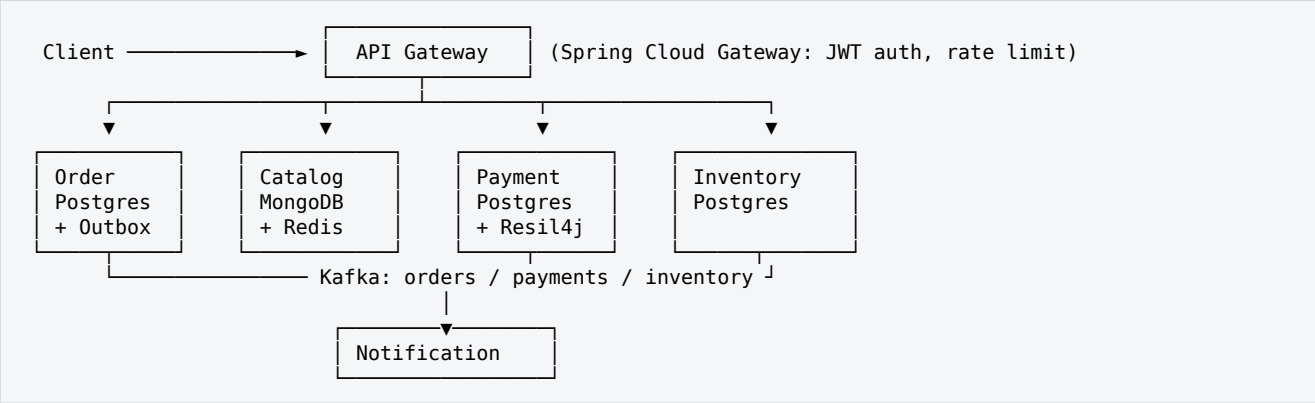
Exit 137 = killed (OOM/liveness); exit 1 = your app threw.

Check for understanding

Q1. After deploying v1.5, pods show `CrashLoopBackOff` with 6 restarts. `kubectl logs --previous` shows: `Failed to configure a DataSource: 'url' attribute is not specified`. What's the likely cause and the first two actions you take?

Capstone: E-Commerce Order Management System

Put every module to work by building a production-style system, one milestone at a time. Finish each milestone with tests passing before moving on.



Service	Store	Why this choice
Order	PostgreSQL	Money & state transitions need ACID, constraints, `@Version`
Catalog	MongoDB + Redis	Varied product attributes; read-heavy, so cache-aside with jitter
Payment	PostgreSQL	Ledger integrity; Resilience4j around the external gateway
Inventory	PostgreSQL	Atomic `UPDATE ... WHERE stock > 0` for flash sales
Sessions / cart	Redis	Short-lived, high-frequency, TTL-based

Milestones

#	Milestone	Modules applied	Done when…
1	Order Service: Spring Boot 3 + PostgreSQL + Flyway; REST CRUD; layered (controller → service → repository); DTO records; validation; global `@RestControllerAdvice`	1, 2, 5	Integration tests with Testcontainers pass; `EXPLAIN` shows index scans
2	Catalog Service: MongoDB documents, ESR indexes, search & aggregation	3	Category filter + sort served by IXSCAN
3	Redis cache-aside on product reads; TTL jitter; null caching	4	p95 latency drops; DB QPS falls on load test
4	Payment client with timeouts, Retry + CircuitBreaker + fallback	5	Killing the fake gateway doesn't take Order down
5	Kafka events with Outbox; choreographed Saga with compensations; idempotent consumers; DLT	6	Kill a consumer mid-flow → no duplicates, no lost orders
6	Dockerfiles (multi-stage), docker-compose for local; K8s manifests with probes, resources, ConfigMaps/Secrets	7	Rolling update with zero failed requests
7	Observability: Micrometer + Prometheus, OpenTelemetry tracing, correlation IDs in logs	All	One trace shows an order across all services

Beginner mistakes to watch for in the project

- **Connection leaks / pool exhaustion:** set `spring.datasource.hikari.leak-detection-threshold=20000` in dev; never hold a transaction across a remote call.
- **N+1 queries:** enable SQL logging in dev and use `JOIN FETCH` / `@EntityGraph` for lists.

- **Missing indexes:** run `EXPLAIN ANALYZE` on every query in the hot path.
- **Kafka rebalancing storms:** keep listeners fast; watch for repeated "Revoking partitions" logs.
- **Wrong probes:** no DB checks in liveness; add a startup probe.

Suggested prompt for a step-by-step build session: "Act as a Senior Tech Lead and guide me through Milestone 1: setting up the Spring Boot Order Service with PostgreSQL."

Answer Key

Try each question yourself before reading the answer. Struggling first is how it sticks.

Module 1: Java Core Internals

1.1 JVM Memory & Garbage Collection

`temp` is a reference on the thread's **stack**. Each `Order` object is on the **heap** (Eden).

Yes. When `temp` is reassigned, the old object has no references left, so it is unreachable.

Minor GC. They die young in Eden and are never promoted. That is exactly what the generational design optimises for.

1.2 Collections Internals: ArrayList, HashMap, ConcurrentHashMap

Capacity doubles from 16 until it exceeds 1,000,000 / 0.75 ≈ 1.33M, so 16 → 2,097,152 is **17 resizes**, each rehashing every entry. Pre-size with `new HashMap<>(1_400_000)` (or `HashMap.newHashMap(1_000_000)` in Java 19+).

```
public record OrderKey(String customerId, LocalDate date) {}
```

Records are immutable (final fields) and generate `equals`/`hashCode` from all components, so the hash never changes after insertion.

1.3 Multithreading & Concurrency

`volatile` guarantees visibility, but `count++` is still three steps (read, add, write). Two threads can read the same value before either writes. You need atomicity: `AtomicInteger` or a lock.

The service is a **singleton**, so all request threads share one `ArrayList`: data from different users mixes, and concurrent `add` can corrupt the list or throw. Make it a local variable inside the method.

1.4 Virtual Threads (Java 21+)

Each request mostly waits on I/O. With 200 platform threads, requests queue; with virtual threads all 2,000 can wait concurrently while a handful of carriers do the actual work. The new bottleneck is downstream: HTTP client connection pools, DB pools and the called services' capacity. Configure their limits and timeouts.

Module 2: SQL & Relational Databases

2.1 Indexes: B-Trees, Composite Indexes & EXPLAIN

No. `customer_id` is not the leftmost column, so the index can't seek on it (at best a full index scan). Either add an index on `(customer_id)` or reorder to `(customer_id, status)` if that also serves your other queries. `customer_id` is usually the more selective column anyway.

2.2 ACID Transactions

The DB connection and row locks are held for the full 2 s. Under load this exhausts the connection pool, and if the transaction rolls back after the payment succeeded, money was charged with no order. Restructure:

1) Short transaction: save order as `PENDING_PAYMENT`. 2) Call the gateway outside any transaction (with idempotency key + timeout). 3) Short transaction: mark `PAID` or `FAILED`. For cross-service flows, this becomes a **Saga** (Module 6).

2.3 Isolation Levels & Concurrency Anomalies

The **atomic conditional UPDATE**. With very high contention on one row, optimistic locking makes almost every transaction fail and retry (a retry storm). `UPDATE products SET stock = stock - 1 WHERE id = ? AND stock > 0` is a single statement: the row lock serialises the

buyers briefly, exactly 10 succeed (1 row updated) and the rest see 0 rows and get "sold out". Optimistic locking shines when conflicts are rare.

Module 3: MongoDB & Document Storage

3.1 The Document Model & When to Use MongoDB

(a) **MongoDB**: varied attributes per category, read as whole documents, schema evolves.

(b) **PostgreSQL**: money needs strict ACID, constraints and relational reporting.

(c) **Redis**: short-lived, high-frequency key-value access with TTL (Module 4).

3.2 Embedding vs. Referencing

`posts` document embeds `latestComments` (capped at 20, maintained with `$push` + `$slice: -20`) plus `commentCount`. All comments live in a separate `comments` collection referencing `postId`, with an index on `{postId: 1, createdAt: -1}` for pagination. One read renders the page; no 16 MB risk.

3.3 MongoDB Indexes & Aggregation Pipeline

`{ userId: 1, status: 1, createdAt: -1 }`. `userId` and `status` are equality fields; `createdAt` serves both the sort and the range, so it goes last.

Module 4: Redis Caching

4.1 Redis Fundamentals & Data Structures

A **Sorted Set** per day: `ZINCRBY bestsellers:2026-10-21 <qty> <sku>` on each sale, `ZREVRANGE bestsellers:2026-10-21 0 9 WITHSCORES` to read the top 10, and `EXPIRE` the key after a couple of days.

4.2 The Cache-Aside Pattern

If you evict first, a concurrent reader may miss the cache, read the **old** value from the DB (not yet committed) and re-cache it. The cache then stays stale until TTL. Evicting after commit (or evicting twice: before and after, the "delayed double delete") closes that window.

4.3 Cache Failure Modes: Avalanche, Penetration, Breakdown

All 1M keys expire within seconds of each other, so a wave of misses hits Postgres: a **cache avalanche**. Fix: add jitter, e.g. `Duration.ofHours(24).plusMinutes(random.nextInt(0, 120))`, so expiries spread over two hours.

Module 5: Spring Boot Core & Resiliency

5.1 IoC, Dependency Injection & Auto-Configuration

Jackson's auto-configuration declares its `ObjectMapper` bean with `@ConditionalOnMissingBean`. Your bean already exists, so the condition fails and the auto-config backs off. (Tip: to tweak Boot's mapper rather than replace it, define a `Jackson2ObjectMapperBuilderCustomizer` bean.)

5.2 @Transactional & the Proxy Trap

Separate bean: **yes, saved**. The call goes through `AuditService`'s proxy, which suspends the outer transaction and commits its own. Same class: **no**. Self-invocation bypasses the proxy, so `REQUIRES_NEW` is ignored; the insert runs in the outer transaction and is rolled back with it.

5.3 Resilience4j: Circuit Breaker, Retry, Timeouts & Bulkheads

Within seconds all 200 threads are stuck waiting, so your order service stops answering every endpoint, even ones not needing inventory: a **cascading failure**. With a breaker (+ a short timeout), after ~10 failures it opens, later calls fail fast in microseconds with a fallback (e.g.

"accept order, reserve stock later"), threads stay free, and HALF_OPEN probes detect recovery automatically.

Module 6: Microservices & Event Streaming

6.1 Microservice Architecture Fundamentals

Publish one `OrderPlaced` event. Email and dashboard are clearly **async**: the user shouldn't wait for them and they can catch up later. Stock reservation can be async too, as a step in a **Saga** (next topics). Make it a sync call only if the UI must show "in stock" before confirming, and even then, protect it with a timeout and a circuit breaker.

6.2 Kafka: Topics, Partitions & Consumer Groups

Only **4** consume; 2 sit idle because each partition is assigned to one consumer in a group. To use 6, increase the topic to ≥ 6 partitions. Note: adding partitions changes `hash(key) % n`, so key-to-partition mapping changes for future messages. Plan ahead.

6.3 The Saga Pattern & Transactional Outbox

1. Order: create `PENDING` (compensate: mark `CANCELLED`).
2. Inventory: reserve stock (compensate: release reservation).
3. Payment: charge card (compensate: refund).
4. Order: mark `CONFIRMED`.

Put the step that is **hardest or most costly to compensate** (charging money, since refunds take days and annoy customers) as late as possible, after cheap, reversible steps like stock reservation have succeeded.

6.4 Delivery Guarantees, Idempotency & Dead-Letter Topics

The offset wasn't committed, so the message is **redelivered** and points are added again. Prevent it by storing the `eventId` in a `processed_events` table in the **same DB transaction** as the points update (or use a unique constraint like `(order_id, bonus_type)`), and skip when it already exists.

Module 7: Containerization & Kubernetes

7.1 Docker & Multi-Stage Builds

Docker caches each layer and reuses it while its inputs are unchanged. Copying only `pom.xml` first means the slow dependency download is cached until the pom changes. If you copy `src/` first, every code change invalidates that layer and all dependencies are downloaded again on **every** build (minutes instead of seconds).

7.2 Kubernetes Core Objects: Pod, Deployment, Service, ConfigMap, Secret

The **ReplicaSet** (owned by the Deployment) sees 2 running pods versus the desired 3 and immediately creates a new pod. The Service stops routing to the deleted pod and starts routing to the new one once it becomes Ready.

7.3 Health Probes: Liveness, Readiness & Startup

Liveness starts at 30 s, fails at 30/40/50 s, so at ~50 s the container is **killed and restarted**, before it ever finishes starting. It loops forever (`CrashLoopBackOff`). Fix: add a **startupProbe** (e.g. `periodSeconds: 5`, `failureThreshold: 30`) so liveness only begins after startup succeeds.

7.4 Troubleshooting Pods

The datasource URL isn't reaching the app: the ConfigMap is missing/renamed, the `envFrom` reference changed, or a profile didn't activate. Actions: (1) `kubectl rollout undo deployment/order-service` to restore service immediately; (2) inspect `kubectl describe pod` / `kubectl get configmap order-service-config -o yaml` and compare the env vars with

what v1.5 expects, then fix and redeploy.